

COMP0008: Computer Architecture and Concurrency

Live Session (week 10)

Stefano Vissicchio

References: GPBBHL chapter 10

Warmup

- Self-assess on www.menti.com w/ code 1720 4142
 - would help me try and tune review speed
- Interactions have been great so far
 - let's keep going!
- Check out CW6
 - try to solve it (if you haven't already) or re-check your solutions
 - CW6 is available within the "Formative Courseworks" section on Moodle

Concurrency story so far

- Concurrency enables **superior performance**
 - use of multiple processors, responsiveness, etc.
- **Threads** are the minimal units that OSes schedule to run concurrently
 - threads' atomic actions can be arbitrarily interleaved, reordered, and have effects not universally visible
- Multi-threaded applications are prone to **interference and visibility problems**

General rule: if multiple threads access the same mutable state variable **without appropriate synchronization, your program is broken**

Concurrency story so far (cont'd)

- **Synchronization** restricts the set of possible interleavings of a concurrent program
 - **locks** ensure mutual exclusion of code blocks, and visibility
 - **safe publication** avoids sharing partially constructed objects
- **Appropriate synchronization avoids interference and visibility problems**
 - sacrificing some performance gain
- Is that enough to ensure application correctness?

Even synchronization is not enough

- **Synchronization** restricts the set of possible interleavings of a concurrent program
 - **locks** ensure mutual exclusion of code blocks, and visibility
 - **safe publication** avoids sharing partially constructed objects
- **Appropriate synchronization avoids interference and visibility problems**
 - sacrificing some performance gain
- Is that enough to ensure application correctness?
Unfortunately, **no!**

Even synchronization is not enough

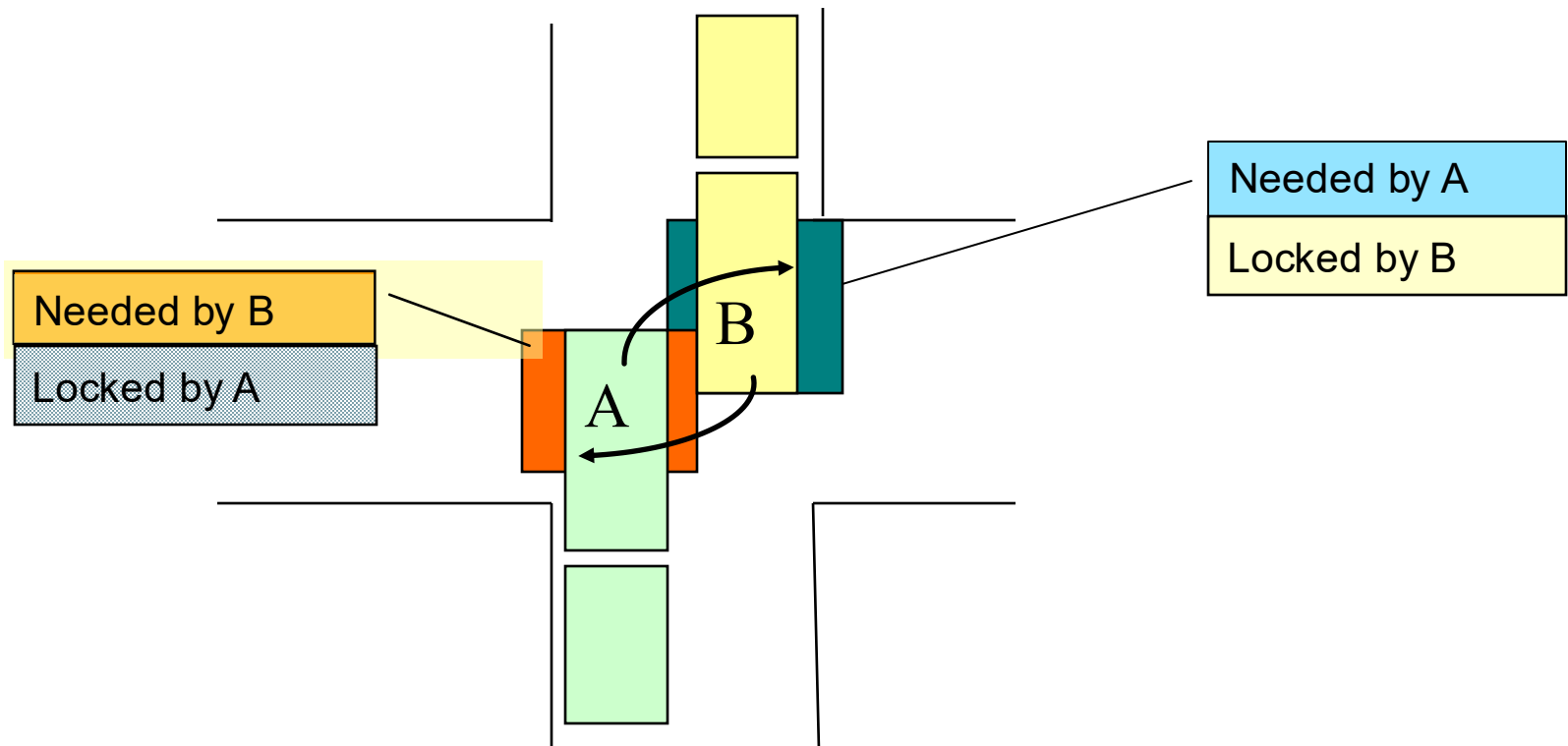
- Correctness typically refers to an expected, useful behaviour of the program
- This entails two properties
 - **safety**: nothing bad happens ← focus so far
 - **liveness**: something good eventually happens
- Does a fully synchronized, thread-safe program ensure liveness too?

Even synchronization is not enough

- Correctness typically refers to an expected, useful behaviour of the program
- This entails two properties
 - **safety**: nothing bad happens ← focus so far
 - **liveness**: something good eventually happens
- Does a fully synchronized, thread-safe program ensure liveness too? Again, **no!**
 - even worse, **safety is generally at odds with liveness**: locking can prevent anything good from happening

Liveness issue #1: Deadlock

- Definition: **all threads are permanently blocked**
 - common case: threads wait forever for each other due to **cyclic dependencies in resource acquisition**
- Real-life analogue: traffic gridlock



The transferMoney example

```
public class Bank {  
    ...  
    public void transferMoney(Account fromAcc,  
                               Account toAcc,  
                               Double amount) {  
        synchronized (fromAcc){  
            synchronized (toAcc){  
                if (fromAcc.getBalance() < amount){  
                    throw new Exception("Insufficient funds");  
                }  
                fromAcc.debit(amount);  
                toAcc.credit(amount);  
            }  
        }  
    }  
}
```

The transferMoney example

```
public class Bank {  
    ...  
    public void transferMoney(Account fromAcc,  
                               Account toAcc,  
                               Double amount) {  
        synchronized (fromAcc){  
            synchronized (toAcc){  
                if (fromAcc.getBalance() < amount){  
                    throw new Exception("Insufficient funds");  
                }  
                fromAcc.debit(amount);  
                toAcc.credit(amount);  
            }  
        }  
    }  
}
```

Is there a deadlock here?
If so, how can a deadlock occur?

The transferMoney example



```
public class Bank {  
    ...  
    public void transferMoney(Account fromAcc,  
                               Account toAcc,  
                               Double amount) {  
        synchronized (fromAcc){  
            synchronized (toAcc){  
                if (fromAcc.getBalance() < amount){  
                    throw new Exception("Insufficient funds");  
                }  
                fromAcc.debit(amount);  
                toAcc.credit(amount);  
            }  
        }  
    }  
}
```

Is there a deadlock here?

Maybe -- depends on how the
method is called by threads! 11

The transferMoney example



```
public class Bank {
    ...
    public void transferMoney(Account fromAcc,
                              Account toAcc,
                              Double amount) {
        synchronized (fromAcc){
            synchronized (toAcc){
                if (fromAcc.getBalance() < amount){
                    throw new Exception("Insufficient funds");
                }
                fromAcc.debit(amount);
                toAcc.credit(amount);
            }
        }
    }
}
```

Is there a deadlock here?

[T1] call transferMoney(A,B,1)

[T1] get lock A

[T2] call transferMoney(B,A,1)

[T2] get lock B

Liveness issue #1: Deadlock examples

- Definition: all threads are blocked (e.g., because of cyclic dependencies in resource acquisition)

Type	Example	Reference
Single object, conflicting methods	LeftRightDeadlock	[GPBBHL 10.1.1]
	Multiple Databases	[GPBBHL 10.1.5]

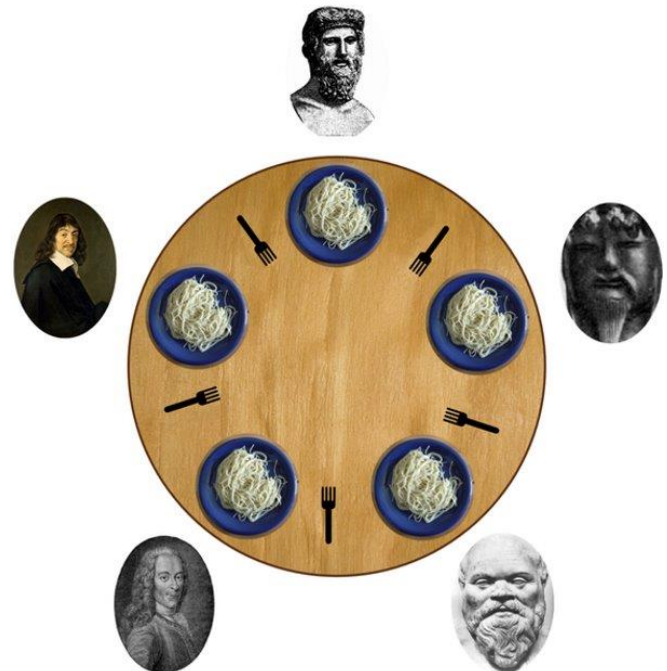
Liveness issue #1: Deadlock examples

- Definition: all threads are blocked (e.g., because of cyclic dependencies in resource acquisition)

Type	Example	Reference
Single object, conflicting methods	LeftRightDeadlock	[GPBBHL 10.1.1]
	Multiple Databases	[GPBBHL 10.1.5]
Single object, conflicting external calls	transferMoney	[GPBBHL 10.1.2]
	Dining Philosophers	[Moodle]

Dining Philosophers

- Dining Philosophers is a classic concurrency problem
 - Formulated by Dijkstra in 1965
 - Employed as test case by many synchronization algorithms
- Problem statement
 - 5 philosophers around a table
 - each philosopher is a thread
 - Each philosopher spends time alternating thinking and eating
 - To eat, each philosopher needs access to 2 forks
 - Only 5 forks on the table
 - each fork is on the left of one philosopher and on the right of another philosopher



Liveness issue #1: Deadlock examples

- Definition: all threads are blocked (e.g., because of cyclic dependencies in resource acquisition)

Type	Example	Reference
Single object, conflicting methods	LeftRightDeadlock	[GPBBHL 10.1.1]
	Multiple Databases	[GPBBHL 10.1.5]
Single object, conflicting external calls	transferMoney	[GPBBHL 10.1.2]
	Dining Philosophers	[Moodle]
Cross objects	Taxi dispatcher	[GPBBHL 10.1.3]

Fixing the transferMoney example

```
public class Bank {  
    ...  
    public void transferMoney(Account fromAcc,  
                               Account toAcc,  
                               Double amount) {  
        synchronized (fromAcc){  
            synchronized (toAcc){  
                if (fromAcc.getBalance() < amount){  
                    throw new Exception("Insufficient funds");  
                }  
                fromAcc.debit(amount);  
                toAcc.credit(amount);  
            }  
        }  
    }  
}
```

How to avoid deadlocks here?

Fixing the transferMoney example

- A possible fix is to enforce a **lock ordering** discipline
 - constraining all threads to get locks in the **same order**
 - hence, avoiding cyclic dependencies

```
public class Bank {  
    ...  
    private void doTransfer(Account fromAcc,  
                             Account toAcc,  
                             Double amount) {  
        if (fromAcc.getBalance() < amount){  
            throw new Exception("Insufficient funds");  
        }  
        fromAcc.debit(amount);  
        toAcc.credit(amount);  
    }  
    ...  
}
```

Fixing the transferMoney example

```
...
public void transferMoneyOrdered(Account fromAcc,
                                Account toAcc,
                                Double amount) {
```

```
    int fromId = fromAcc.getAccountId();
```

```
    int toId = toAcc.getAccountId();
```

return immutable
unique id of account

```
    if (fromId < toId){
```

```
        synchronized (fromAcc){
```

```
            synchronized (toAcc){
```

```
                doTransfer(fromAcc, toAcc, amount);
```

```
            }
```

```
        }
```

```
    }
```

```
    ...
```

```
}
```

```
}
```

otherwise, if toId < fromId, first acquire the toAcc
lock and then the fromAcc lock

Methodological tool: Open calls

- In other cases, deadlocks may be caused by **calling an external method while holding a lock**
- **Open call**: call to an external method (possibly of another object) **with no lock held**
 - we should strive to be completely **agnostic of and robust to the implementation of called methods**
- Using open calls makes it easier to ensure locks are acquired in a consistent order
 - code paths acquiring locks are easier to follow (similar role as encapsulation for thread safety)
- **Caveat**: open calls may induce **loss of atomicity**

Avoiding deadlocks: Summary

- Methodology: identify where resources are acquired, then **ensure no conflicting acquisition or break cycles**

Technique	Example	Reference
Threads never acquire >1 resource at the time	-	[GPBBHL10.2]
Order resource acquisition within individual methods	transferMoneyOrdered	[GPBBHL 10.1.2]
Use open calls across objects	ThreadSafe Taxi dispatcher	[GPBBHL 10.1.4]
Acquire with timeout	transferMoney w/ tryLock	[GPBBHL 10.2.1]
Detect-and-abort	DBMS	[GPBBHL10.1]

Liveness issue #2: Starvation

- Definition: a thread is **perpetually denied access to resources** it needs in order to progress

Resource waited for	Example	Reference
CPU cycles	low-priority threads starved by high-priority ones	[GPBBHL 10.3.1]
Object or Lock	lock held by a thread running an infinite loop	[GPBBHL 10.3.1]
	Readers and Writers	[Moodle]

Light version of starvation: unfair allocation of resources (e.g., CPU), possibly causing poor responsiveness

Liveness issue #3: Livelock

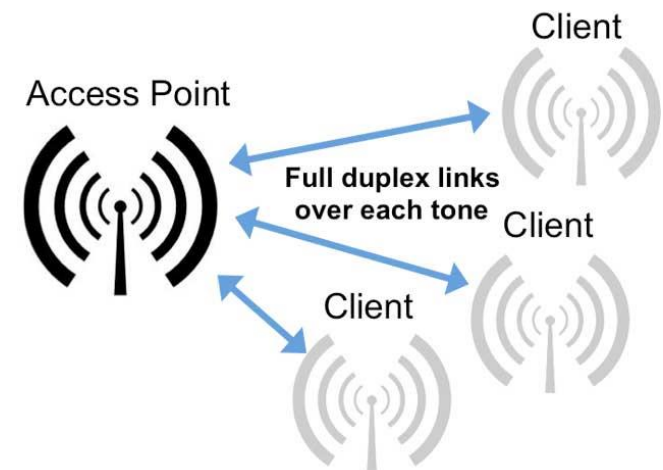
- Definition: despite not blocked, a thread does **not make any useful work**
 - e.g., keeps retrying operation that always fails

Failing operation	Example	Reference
Change state (in response to other thread's state change)	Polite people in a hallway	[GPBBHL 10.3.3]
Faulty error recovery	Poison message problem	[GPBBHL 10.3.3]
	WifiLink	[Moodle]

Typical solution: add randomness to retry mechanism

Livelock example: WifiLink

- Consider a simulation of transmissions on a Wi-Fi link
 - Wi-Fi clients transmit messages to an access point (AP)
 - Messages simultaneously transmitted by clients "collide", and they are not received correctly (i.e., signals mix up)
- Problem formulation
 - 1 AP and N clients
 - The AP waits for messages, and confirms correct receptions
 - Each client sends one message to AP, waits for confirmation, and resends if no confirmation
 - Messages and AP confirmations take time to reach destinations



Reminder: conditional sync

- Problem: what if a thread cannot work on the current state of a monitor (e.g., read empty buffer)?
 - fail / raise an error ← only possibility for single-thread apps
 - wait for another thread to change the monitor's state
- Conditional sync supports the second possibility

PSEUDO-CODE

```

acquire lock
while (predicate) {
    release lock
    wait to be notified
    reacquire lock
}
perform action
notify other threads
release lock
  
```

EXAMPLE

```

public synchronized V read(){
    while(isEmpty()){
        wait();
    }
    V val = doRead();
    notifyAll();
    return val;
}
  
```

Understanding conditional sync

- Mechanism: **condition queues** give threads a way to subscribe to specific updates on a given condition
 - waiting threads form a so-called **wait set**
- Condition queues offer methods to
 - **wait()** allows threads to enter a condition queue
 - **notify()** wakes up one thread in the wait set
 - **notifyAll()** wakes up *all* threads in the wait set
- Important: notifications signal that "something has changed", **not what** has changed
 - threads must re-check condition when woken up

Understanding conditional sync

- In Java, **each object** can act as a condition queue
 - this.wait() makes a thread wait until this.notifyAll() is called
- Which lock and conditional queue to use?
 - the **condition predicate** is a condition on state variables
 - before testing the condition predicate, we must hold the **lock guarding the corresponding state variables**
 - we must also hold that lock when calling wait and notification methods on the condition queue
 - additionally, **lock object == condition queue object**
- NB: we should think of **entry and exit protocols**

Starvation example: ReadersWriters

- Inspired by Readers and Writers [Magee & Kramer ch.7]
 - **Shared database (DB)** accessed by two types of threads: Readers that read the DB, and Writers that update it.
 - **Writers must have exclusive access to the DB**, while any number of Readers may simultaneously access it.



Liveness issue #4: Incorrect conditional sync

- Notably, **missed signals**: a thread keeps waiting for a condition that is already true, or was true in the past
- Incorrectness often stems from inappropriate use of conditional synchronization and/or code bugs
 - example1: condition predicate not checked before waiting
 - example2: the thread is not woken up from a wait
- Let's revisit these mechanisms...

Conditional sync in action

- Please, connect to www.menti.com
- There, enter code 1720 4142



Please enter the code

Submit

The code is found on the screen in front of you